

TP Modul 11

Nama : William Peter Vanx Najohan

NIM : 103032400084

Kelas : IT-48-01

Kode Asprak : APY

tree.h

```
1  #ifndef TREE_H_INCLUDED
2  #define TREE_H_INCLUDED
3  #include <iostream>
4
5  using namespace std;
6
7  typedef int infotype;
8  typedef struct node *adrNode;
9
10 struct node {
11     infotype info;
12     adrNode left;
13     adrNode right;
14 };
15
16 void createTree_103032400084(adrNode &root);
17 adrNode createNode_103032400084(infotype x);
18 void insertNode_103032400084(adrNode &root, adrNode p);
19 adrNode searchNode_103032400084(adrNode root, infotype x);
20 void displayTree_103032400084(adrNode root);
21 int countNodes_103032400084(adrNode root);
22 infotype getMinValue_103032400084(adrNode root);
23 infotype getMaxValue_103032400084(adrNode root);
24
25 #endif // TREE_H_INCLUDED
26
```

tree.cpp

```

1  #include "tree.h"
2
3  void createTree_103032400084 (adrNode &root) {
4      root = nullptr;
5  }
6
7  adrNode createNode_103032400084 (infotype x) {
8      adrNode p = new node;
9
10     p -> info = x;
11     p -> left = nullptr;
12     p -> right = nullptr;
13
14     return p;
15 }
16
17 void insertNode_103032400084 (adrNode &root, adrNode p) {
18     if (root == nullptr) {
19         root = p;
20     } else if (p -> info < root -> info) {
21         insertNode_103032400084 (root -> left, p);
22     } else {
23         insertNode_103032400084 (root -> right, p);
24     }
25 }
26
27 adrNode searchNode_103032400084 (adrNode root, infotype x) {
28     if (root == nullptr) {
29         return nullptr;
30     } else if (x == root->info) { //line tambahan yang tidak ada di modul

```

```

31         return root; //line tambahan yang tidak ada di modul
32     } else if (x < root -> info) {
33         return searchNode_103032400084 (root -> left, x);
34     } else {
35         return searchNode_103032400084 (root -> right, x);
36     }
37
38     return root; //line ini sudah tidak diperlukan
39 }
40
41 void displayTree_103032400084 (adrNode root) {
42     if (root != nullptr) {
43         displayTree_103032400084 (root -> left);
44         cout << root -> info << " ";
45         displayTree_103032400084 (root -> right);
46     }
47 }
48
49 int countNodes_103032400084 (adrNode root) {
50     if (root == nullptr) {
51         return 0;
52     } else {
53         return 1 + countNodes_103032400084 (root -> left) + countNodes_103032400084 (root -> right);
54     }
55 }
56
57 infotype getMinValue_103032400084 (adrNode root) {
58     adrNode x = root;
59
60     if (x == nullptr) {

```

```

61         cout << "root kosong";
62         x -> info = 0;
63     }
64
65     while (x -> left != nullptr) {
66         x = x -> left;
67     }
68
69     return x -> info;
70 }
71
72 infotype getMaxValue_103032400084(adrNode root) {
73     adrNode x = root;
74
75     if (x == nullptr) {
76         cout << "root kosong";
77         x -> info = 0;
78     }
79
80     while (x -> right != nullptr) {
81         x = x -> right;
82     }
83
84     return x -> info;
85 }
86

```

beberapa line yang dimodifikasi:

line 30 & line 31

penjelasan: 2 line tersebut adalah line tambahan yang tidak ada didalam modul, karena menurut saya sendiri perlu ada 2 line tambahan tersebut agar fungsi searchNode dapat mengembalikan nilai node yang dicari.

main.cpp

```

1  #include "tree.h"
2
3  int main() {
4      int nodeCari, totNode, minNode, maxNode, inNode;
5      adrNode root, inAdrNode, x;
6
7      createTree_103032400084(root);
8
9      for (int i = 0; i < 7; i++) {
10         cout << "Masukkan node: ";
11         cin >> inNode;
12         inAdrNode = createNode_103032400084(inNode);
13         insertNode_103032400084(root, inAdrNode);
14     }
15     cout << endl;
16
17     cout << "Masukkan nilai dari node yang ingin dicari: ";
18     cin >> nodeCari;
19
20     x = searchNode_103032400084(root, nodeCari);
21
22     if (x != nullptr) {
23         cout << "Node dengan nilai " << nodeCari << " ditemukan!";
24     } else {
25         cout << "Node dengan nilai " << nodeCari << " tidak ditemukan!";
26     }
27     cout << endl << endl;
28
29     cout << "Print BST (inorder traversal): ";
30     displayTree_103032400084(root);

```

```

31     cout << endl;
32
33     totNode = countNodes_103032400084(root);
34     cout << "Jumlah node: " << totNode;
35     cout << endl;
36
37     minNode = getMinValue_103032400084(root);
38     maxNode = getMaxValue_103032400084(root);
39
40     cout << "Nilai terkecil: " << minNode;
41     cout << endl;
42     cout << "Nilai terbesar: " << maxNode;
43
44     return 0;
45 }
46

```

output

```

Masukkan node: 50
Masukkan node: 30
Masukkan node: 70
Masukkan node: 20
Masukkan node: 40
Masukkan node: 60
Masukkan node: 80

Masukkan nilai dari node yang ingin dicari: 60
Node dengan nilai 60 ditemukan!

Print BST (inorder traversal): 20 30 40 50 60 70 80
Jumlah node: 7
Nilai terkecil: 20
Nilai terbesar: 80

```